

ME315 - Lab 03

Maria Luiza dos Santos de Queiroz - RA: 183314

Dados relacionais e operações do tipo *join*

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.
 - Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
 - Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;

```
library(readr)
library(dplyr)

#####

airlines <- read_csv('airlines.csv')

#####

airports <- read_csv('airports.csv',
  col_selec = c('IATA_CODE', 'CITY', 'STATE',
    'LATITUDE', 'LONGITUDE'))

#####

getStats <- function(input, pos){
  input %>%
    filter(substr(DESTINATION_AIRPORT, 1, 1) != "1") %>%
```

```

na.omit() %>%
group_by(DESTINATION_AIRPORT) %>%
summarise(
  atrasos = sum(ARRIVAL_DELAY),
  voos = n(),
  .groups = "drop"
)
}

flights <- read_csv_chunked(
  'flights.csv',
  callback = DataFrameCallback$new(getStats),
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```

media_atraso <- flights %>%
group_by(DESTINATION_AIRPORT) %>%
summarise(
  atrasos_totais = sum(atrasos),
  voos_totais = sum(voos)
) %>%
mutate(atraso_medio = atrasos_totais/voos_totais)
media_atraso

```

```

# A tibble: 322 x 4
  DESTINATION_AIRPORT atrasos_totais voos_totais atraso_medio
  <chr>                <dbl>         <int>         <dbl>
1 ABE                  12874           2220           5.80
2 ABI                   9077           2224           4.08
3 ABQ                 106415          18953           5.61
4 ABR                  -2227            657          -3.39
5 ABY                   7501            864           8.68
6 ACK                  2393            487           4.91

```

```

7 ACT                16392        1541        10.6
8 ACV                12259        1267         9.68
9 ACY                41744        3530        11.8
10 ADK                -284         89        -3.19
# i 312 more rows

```

2. Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

- Identifique a coluna que é a chave na tabela `flights`;
- Identifique a coluna que é a chave na tabela `airports`;
- Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?

```

flights %>%
  anti_join(airports,
            by = c("DESTINATION_AIRPORT" = "IATA_CODE"))

```

```

# A tibble: 0 x 3
# i 3 variables: DESTINATION_AIRPORT <chr>, atrasos <dbl>, voos <int>

#Todos os aeroportos presentes em `flights` estão presentes em `airports`.

```

- Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
- Armazene a tabela resultante no objeto `flights`.

```

#coluna DESTINATION_AIRPORT em flights conecta-se com IATA_CODE em airports

flightss <- flights %>%
  left_join(airports,
            by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  select(DESTINATION_AIRPORT, CITY, STATE)
flightss

```

```

# A tibble: 1,874 x 3
  DESTINATION_AIRPORT CITY      STATE
  <chr>              <chr>    <chr>
1 ABE                Allentown PA
2 ABI                Abilene   TX
3 ABQ                Albuquerque NM
4 ABR                Aberdeen  SD
5 ABY                Albany    GA

```

```

6 ACT                Waco                TX
7 ACV                Arcata/Eureka CA
8 ACY                Atlantic City NJ
9 ADK                Adak                AK
10 ADQ               Kodiak                AK
# i 1,864 more rows

```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```

airports %>% group_by(STATE) %>%
  summarise(quantia_aeroportos = n()) %>%
  arrange(desc(quantia_aeroportos))

```

```

# A tibble: 54 x 2
  STATE quantia_aeroportos
  <chr>          <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
5 MI              15
6 NY              14
7 CO              10
8 MN               8
9 MT               8
10 NC              8
# i 44 more rows

```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.
- Carregue o pacote `leaflet`;
 - Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
 - Armazene o grafico em b) numa variável chamada `this`;
 - Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```

library(leaflet)

mapa <- flights %>%
  mutate(atraso_medio = atrasos/voos) %>%
  left_join(

```

```

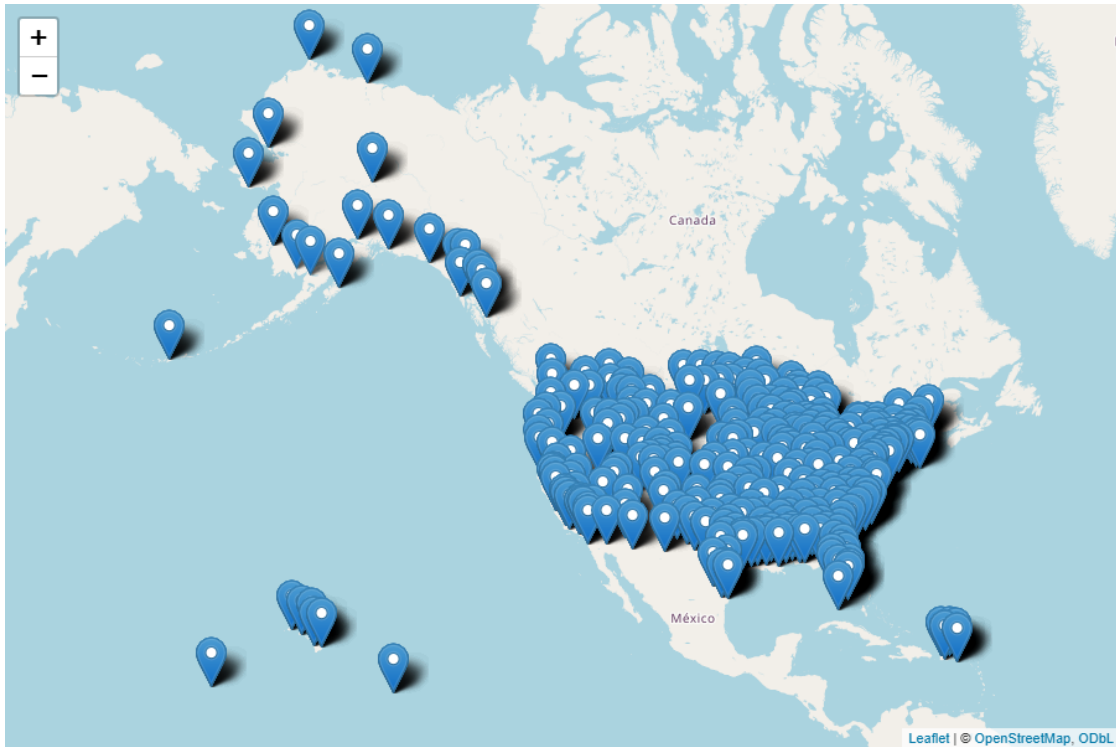
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  filter(!is.na(LATITUDE), !is.na(LONGITUDE))

this <- leaflet(mapa) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~paste0(
      DESTINATION_AIRPORT, " - ", CITY, "/", STATE,
      "<br>Atraso médio: ", round(atraso_medio, 1), " min"
    )
  )

library(htmlwidgets)
library(webshot2)

saveWidget(this, "mapa_temp.html",
            selfcontained = TRUE)
webshot2::webshot("mapa_temp.html", file = "mapa.png",
                  vwidth = 900, vheight = 600)

```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 22:52:10 -03"
```